

WAD Chat

LLM Chat Application

Assignment	Web Application Development
Student ID	ISU520196_3
Date	May 2026
Repository	github.com/brosjsy/llm_assignment_ISU520196_3

1. Project Overview

WAD Chat is a chat application that works like ChatGPT, but runs a local language model entirely on your CPU with no cloud API needed. Users can sign up with a username and password or log in through GitHub. Once inside, they can create conversation threads, ask questions, and get AI responses that stream back to them word by word in real time.

Technologies Used

Layer	Technology	Purpose
Web framework	FastAPI 0.115	Async API and HTML page routes
Database	PostgreSQL 16	Stores users, chats and messages
Migrations	Alembic 1.14	Tracks and applies database schema changes
Session store	Redis 7	Holds refresh token sessions and caches message history
Auth tokens	JWT (python-jose)	Short-lived access tokens and long-lived refresh tokens
Social login	GitHub OAuth 2.0	Lets users sign in with their GitHub account
Password hash	Argon2 (passlib)	Stores passwords securely
LLM runtime	llama-cpp-python	Runs a local GGUF model on CPU
Templating	Jinja2	Server-rendered HTML pages
HTTP client	httpx	Makes async requests to GitHub during OAuth

Architecture

The application uses server-rendered HTML pages, so it follows an MVC pattern (Model, View, Controller):

Models are SQLAlchemy ORM classes that map to database tables.

Views are Jinja2 HTML templates that the server fills in before sending to the browser.

Controllers are FastAPI route handlers. Each one does one thing only: it calls a single method on a service class and returns the result. No business logic lives in the router files.

How JWT and Redis Work Together

When a user logs in, registers, or signs in via GitHub, the server creates an access token (valid for 30 minutes) and a refresh token (valid for 30 days). A unique session ID is saved in Redis with the same 30-day expiry.

Protected pages and API routes read the JWT from either the Authorization header or an httponly cookie set at login.

When the access token expires, the client sends the refresh token. The server checks Redis, deletes the old session, and issues a fresh token pair.

Logging out immediately deletes the Redis entry, so the refresh token stops working right away even if it has not expired yet.

Bonus Features

Streaming output - tokens appear on screen one by one via Server-Sent Events. A blinking cursor shows the response is still being generated.

Message cache - each chat's message list is cached in Redis for 5 minutes to reduce database reads. The cache clears automatically when a new message is added.

Auto title - the first message in a chat becomes the chat title automatically.

2. Code Structure

The project is split into feature packages inside the `app/` folder. Each package contains only files that belong to that feature. Shared concerns like the database connection, Redis client, and settings sit directly under `app/` so every package can import them.

```
app/  
auth/  
depends.py FastAPI dependency that reads the JWT and returns the current user  
router.py Auth controllers - one service call per endpoint  
schemas.py Pydantic request and response models for auth  
security.py Password hashing with Argon2 and JWT creation / decoding  
service.py All auth logic: register, login, refresh, logout, GitHub OAuth  
chats/  
models.py SQLAlchemy ORM models: Chat and Message  
router.py Chat controllers - one service call per endpoint  
schemas.py Pydantic request and response models for chats  
service.py Chat CRUD operations and LLM turn logic (ask, ask_stream)  
llm/  
service.py LLMService class: loads model once, generates text, builds prompts  
users/  
models.py SQLAlchemy ORM model: User  
config.py Reads all settings from the .env file using Pydantic BaseSettings  
database.py Creates the async SQLAlchemy engine and session factory  
main.py Creates the FastAPI app and registers all routers  
redis_client.py A single shared async Redis client used across the app  
  
templates/  
base.html Shared HTML shell with stylesheet link  
auth/login.html Login and Register tabs with GitHub OAuth button  
chats/list.html Sidebar with chat list and welcome screen  
chats/detail.html Chat thread with live SSE streaming input  
  
static/css/style.css Dark theme styles using CSS custom properties  
alembic/versions/ Database migration scripts  
docker-compose.yml Runs PostgreSQL 16 and Redis 7 locally  
requirements.txt All Python dependencies with pinned versions
```

Key Files Explained

app/auth/service.py

This is where all authentication logic lives. It handles registration, login, token refresh, logout, and the full GitHub OAuth flow including the HTTP exchange with GitHub's API. The `_issue_tokens` method creates the JWT pair and saves the session to Redis.

app/chats/service.py

This service owns both chat management and LLM communication. The `ask()` method handles a full non-streaming turn: it validates the chat, auto-titles it if needed, saves the user message, fetches cached history, calls the LLM, and saves the reply. The `ask_stream()` method does the same thing but yields SSE-formatted tokens one at a time and saves the complete response when streaming finishes.

app/llm/service.py

A small wrapper around llama-cpp-python. The model is loaded once when the first request arrives and reused for every request after that. It provides `generate()` for a full response, `generate_streaming()` for token-by-token output, and `build_prompt()` to format the last six messages into a dialogue prompt.

app/auth/depends.py

A FastAPI dependency injected into every protected route. It looks for the JWT in the Authorization header first, then falls back to the `access_token` cookie. If the token is missing or invalid it raises a 401 error.

3. API Structure

The API is built with FastAPI. All protected endpoints accept the JWT either in the Authorization: Bearer header or as an httponly cookie. You can explore and test every endpoint interactively at /docs (Swagger UI).

Authentication Endpoints (prefix: /auth)

Method	Path	Auth	Description
POST	/auth/register	No	Create a new account with login and password. Returns a token pair. Returns 400 if email or password is invalid.
POST	/auth/login	No	Sign in and receive an access token and refresh token via cookies. Returns 400 if email or password is invalid.
POST	/auth/refresh	No	Exchange a refresh token for a new token pair. The old session in Redis is deleted.
POST	/auth/logout	Yes	Delete the session from Redis and clear the auth cookies.
GET	/auth/github	No	Redirect the browser to GitHub to start the OAuth flow.
GET	/auth/github/callback	No	Receive the GitHub code, exchange it for a user profile, create or find the user, and log them in.

Chat Endpoints (JWT required for all)

Method	Path	Auth	Description
GET	/api/chats	Yes	Return a list of all chats belonging to the logged-in user.
POST	/api/chats	Yes	Create a new chat. Accepts a title in the request body.
GET	/api/chats/{id}	Yes	Return a single chat with its full message history. Returns 404 if not found.
DELETE	/api/chats/{id}	Yes	Delete a chat and all its messages. Clears the Redis cache for that chat.
POST	/api/chats/{id}/ask	Yes	Send a message and get a complete LLM reply in one response.
GET	/api/chats/{id}/stream	Yes	Stream the LLM reply token by token using Server-Sent Events. Pass the chat ID in the path.

Page Routes (browser, cookie auth)

Method	Path	Auth	Description
GET	/	No	Login and register page. Redirects to /chats if already logged in.
GET	/chats	Yes	Main chat list page with sidebar and welcome screen.
GET	/chats/{id}	Yes	Chat conversation page with the message thread and streaming input box.

Request and Response Schemas

RegisterRequest: login (string), password (string), email (string, optional)

LoginRequest: login (string), password (string)

RefreshRequest: refresh_token (string)

TokenResponse: access_token (string), token_type: bearer

ChatCreate: title (string, defaults to New Chat)

ChatResponse: id, title, created_at

ChatDetail: id, title, created_at, messages (list of MessageResponse)

MessageResponse: id, role, content, created_at

AskRequest: content (string)

4. Database Schema

PostgreSQL is the main database. All schema changes go through Alembic migrations so the schema history is tracked in version control. The app connects using SQLAlchemy's async ORM with the asyncpg driver.

Table: users

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-generated primary key
login	VARCHAR(150)	UNIQUE NOT NULL	Username, must be unique
email	VARCHAR(255)	NULL	Optional email address
hashed_password	VARCHAR(255)	NULL	Argon2 hash. NULL for GitHub-only accounts
github_id	VARCHAR(100)	UNIQUE NULL	GitHub user ID. NULL for password accounts
is_active	BOOLEAN	NOT NULL	Can be used to disable accounts. Defaults to true
created_at	TIMESTAMP	NOT NULL	When the account was created (UTC)

Table: chats

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-generated primary key
title	VARCHAR(255)	NOT NULL	Chat title, auto-set from the first message
user_id	INTEGER	FK users.id CASCADE	The owner. Deleting the user deletes their chats
created_at	TIMESTAMP	NOT NULL	When the chat was created (UTC)

Table: messages

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-generated primary key
chat_id	INTEGER	FK chats.id CASCADE	The parent chat. Deleting the chat deletes its messages
role	VARCHAR(20)	NOT NULL	Either user or assistant
content	TEXT	NOT NULL	The full message text
created_at	TIMESTAMP	NOT NULL	When the message was saved (UTC)

Relationships

One user can have many chats. Deleting a user removes all their chats.

One chat can have many messages. Deleting a chat removes all its messages.

Redis Keys

Redis is used for two things alongside the relational database:

Key	Value	TTL	Purpose
refresh:{session_id}	user ID (string)	30 days	Refresh token session. Deleted on logout or token rotation.
chat_messages:{id}	JSON message list	5 minutes	Cached message history. Cleared when a new message is

5. User Interface

The UI is built with Jinja2 server-rendered templates and styled with a dark theme. There is no frontend framework. The browser uses the Fetch API for form submissions and the native EventSource API to receive streaming tokens from the server.

Chat Conversation - Streaming Response

This is the main chat view. The user's messages appear on the right in purple bubbles. The AI response appears on the left and streams in word by word while it is being generated. The first message automatically becomes the chat title shown in the header and sidebar.

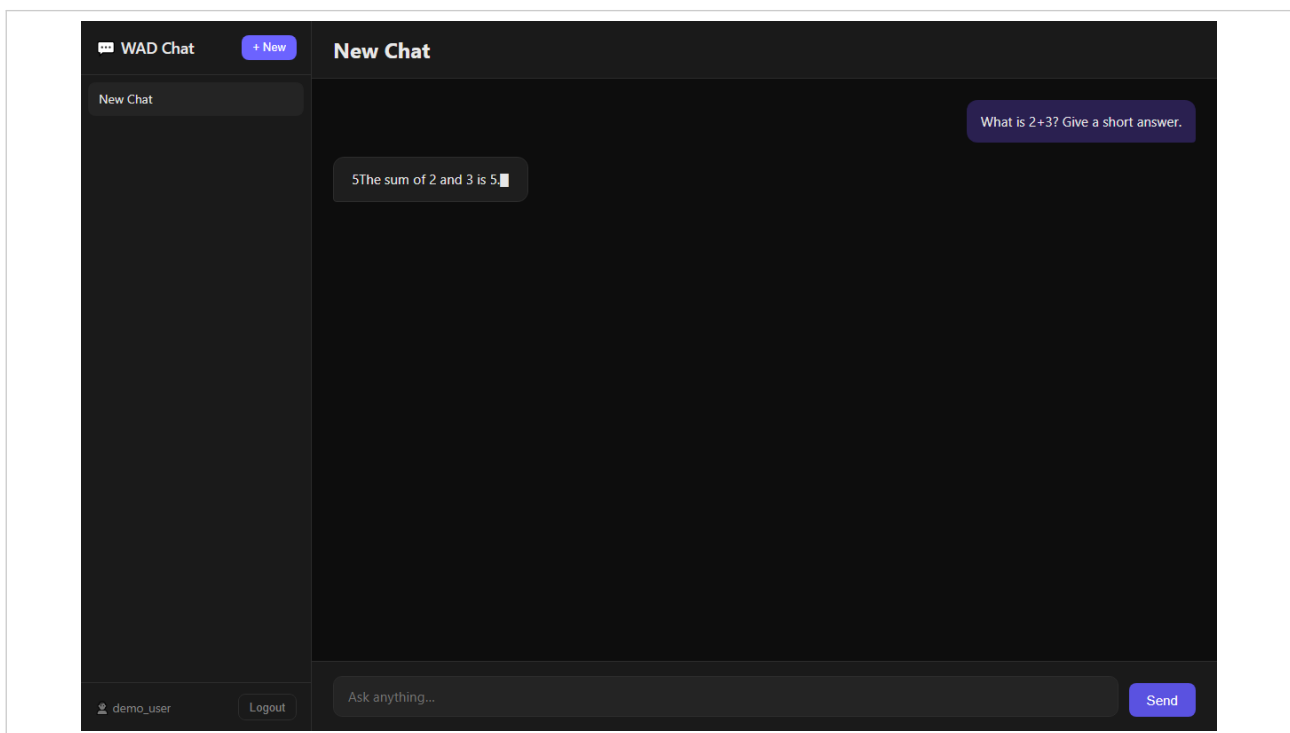


Figure 1 - Active chat showing a streamed AI response

Chat List

After logging in, the user lands on the chat list page. The sidebar on the left shows all previous conversations with their auto-generated titles. Clicking any chat opens it. The logout button sits at the bottom of the sidebar next to the username.

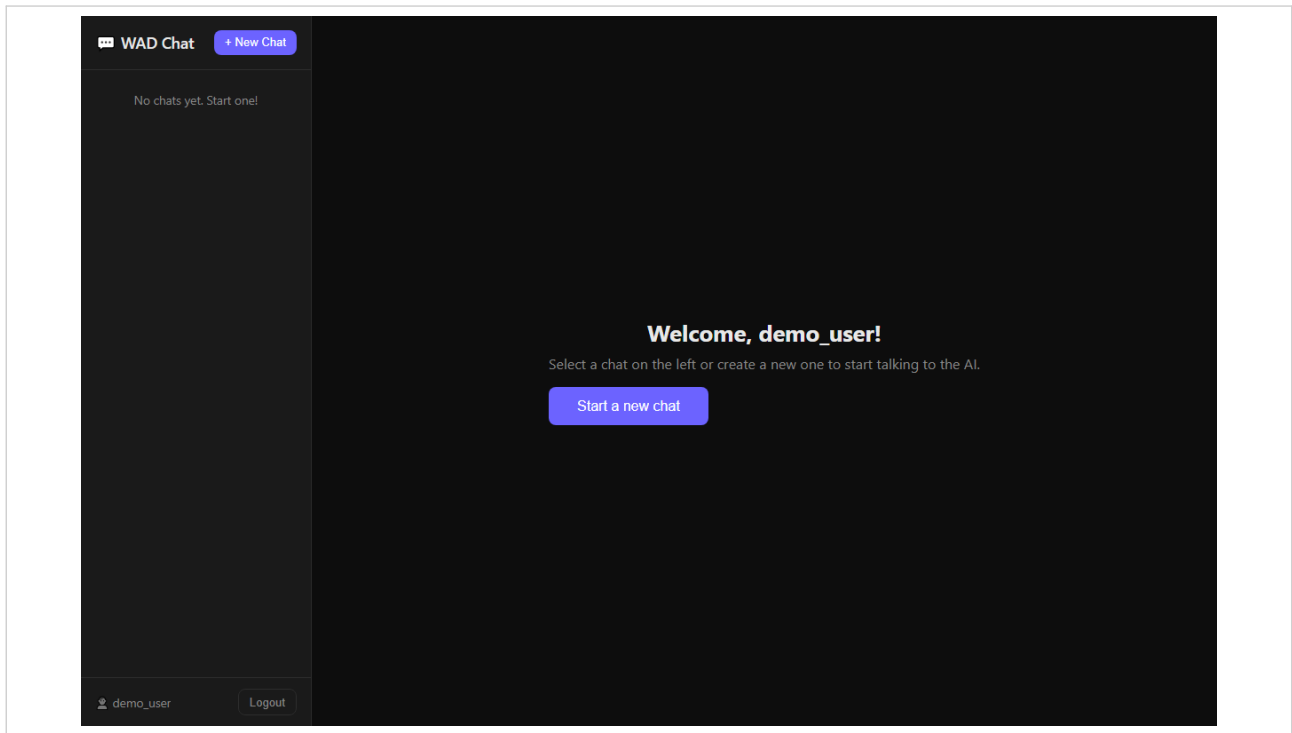


Figure 2 - Chat list with sidebar and welcome screen

Login Page

The login page is the first thing a new visitor sees. It shows a dark card centred on the screen with a Login tab and a Register tab. The Login tab has a username field, a password field, a sign in button, and a Continue with GitHub button below the divider for OAuth login.

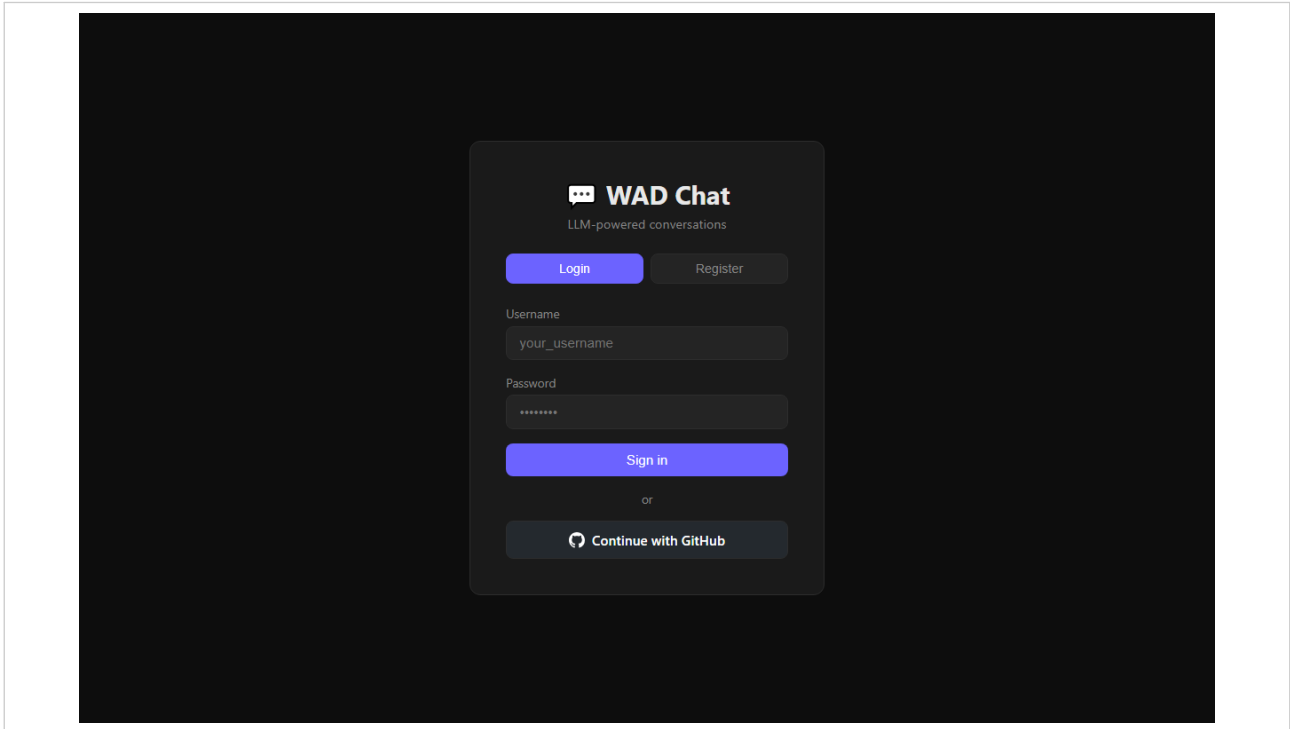


Figure 3 - Login page with GitHub OAuth button

Register Page

Switching to the Register tab reveals a third field for an optional email address. After a successful registration the server sets the auth cookies and redirects the browser straight to the chat list.

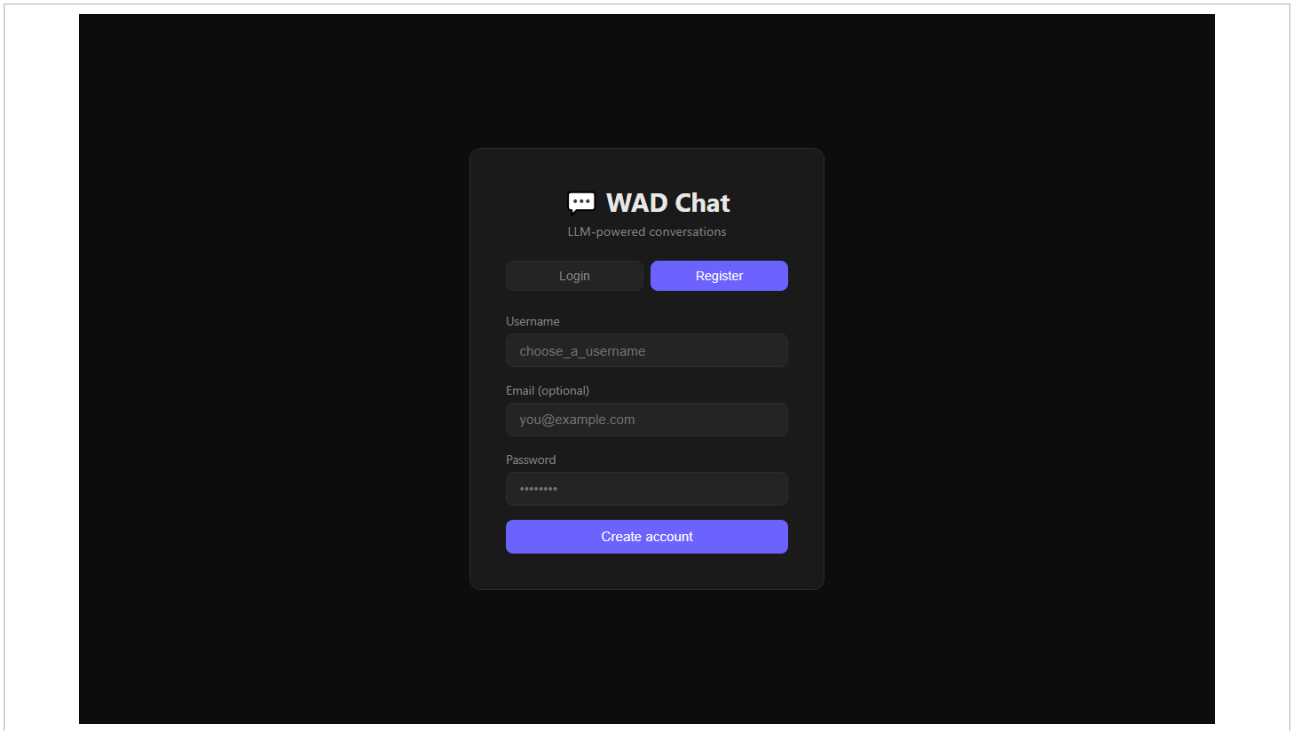


Figure 4 - Register tab with username, optional email and password

API Documentation (Swagger UI)

FastAPI generates interactive API documentation automatically at /docs. Every endpoint is listed with its method, path, request body schema, and response schema. You can expand any endpoint and click Try it out to send a real request directly from the browser without needing a separate API client.

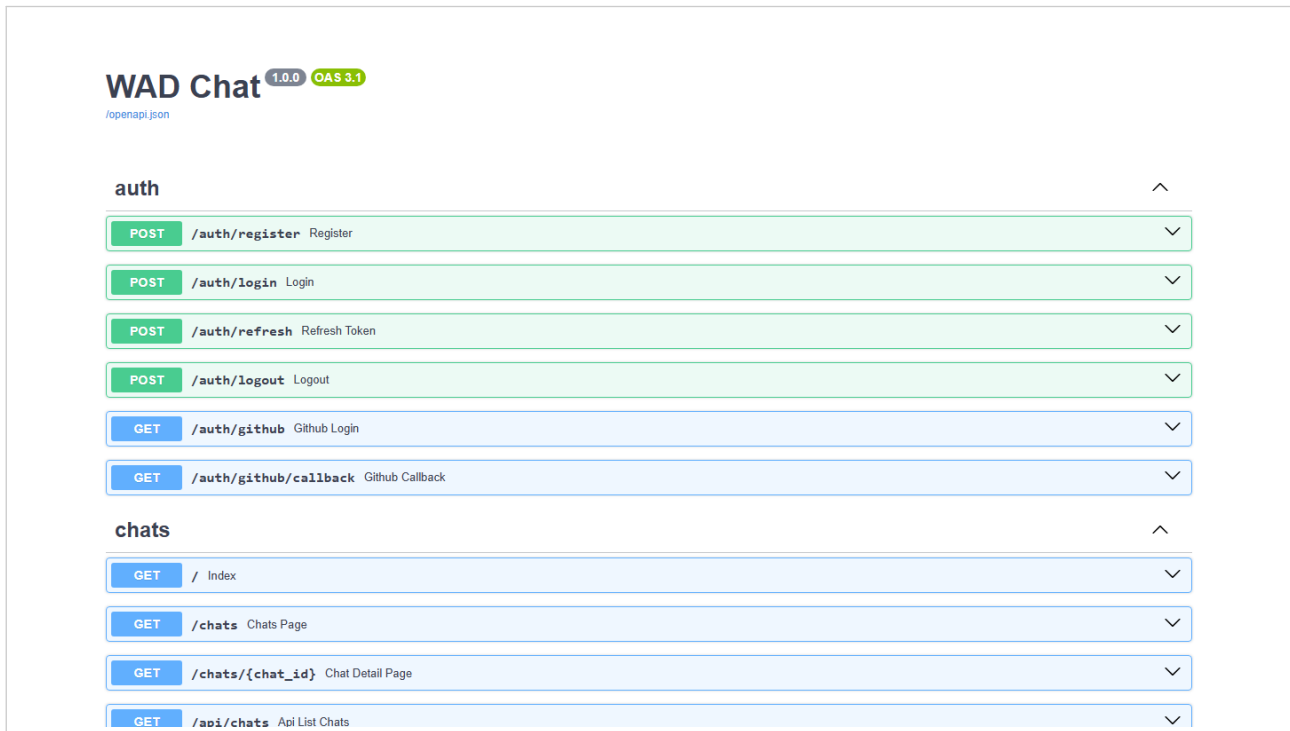


Figure 5 - Swagger UI at /docs listing all API endpoints

6. How to Run the Application

Follow the steps below to get the application running on your machine.

What you need before starting

Make sure these are installed on your machine before you begin:

```
Python 3.11 or newer - https://python.org/downloads  
Docker Desktop - https://docs.docker.com/get-docker/  
Git - https://git-scm.com  
A GGUF model file (any quantised LLaMA, Qwen or Mistral model from Hugging Face)
```

Step 1 - Clone the repository

Open a terminal and run:

```
git clone https://github.com/brosjsy/llm_assignment_ISU520196_3.git  
cd llm_assignment_ISU520196_3
```

Step 2 - Start the database and Redis

Docker Desktop must already be open, then run:

```
docker-compose up -d
```

Step 3 - Create a virtual environment

This keeps the project dependencies separate from your system Python.

```
python -m venv venv  
# Windows:  
venv\Scripts\activate  
# macOS and Linux:  
source venv/bin/activate
```

Step 4 - Install dependencies

With the virtual environment active, run:

```
pip install -r requirements.txt
```

Step 5 - Install llama-cpp-python on Windows (if Step 4 fails for this package)

Use the pre-built CPU wheel instead:

```
pip install llama-cpp-python --extra-index-url  
https://abetlen.github.io/llama-cpp-python/whl/cpu
```

Step 6 - Add the model file

Copy your GGUF model file into the project root folder and name it model.gguf, or set MODEL_PATH in the .env file to point to wherever the file is.

Step 7 - Set up the environment file

Copy the example file and open it in a text editor:

```
cp .env.example .env
```

Step 8 - Fill in the .env file

Set the values shown below. The database and Redis URLs work as-is if you used the docker-compose file in Step 2.

Variable	Example / Default	What it does
DATABASE_URL	postgresql+asyncpg://postgres:postgres@postgres:5432/chat	PostgreSQL database URL
REDIS_URL	redis://localhost:6379	Redis connection URL
SECRET_KEY	change-this-to-a-long-random-string	Signs the JWT tokens. Change this in any real deployment.
ACCESS_TOKEN_EXPIRE_MINUTES	30	How long an access token lasts
REFRESH_TOKEN_EXPIRE_DAYS	30	How long a refresh token lasts (also the Redis TTL)
GITHUB_CLIENT_ID	from GitHub Developer Settings	Only needed if you want GitHub login
GITHUB_CLIENT_SECRET	from GitHub Developer Settings	Only needed if you want GitHub login
GITHUB_REDIRECT_URI	http://localhost:8000/auth/github/callback	Must match the callback URL in your GitHub OAuth app
MODEL_PATH	model.gguf	Path to your GGUF model file

GitHub OAuth setup (only needed for GitHub login)

Go to <https://github.com/settings/developers> and click New OAuth App.

Set Homepage URL to <http://localhost:8000>

Set Authorization callback URL to <http://localhost:8000/auth/github/callback>

Copy the Client ID and Client Secret into your .env file.

Step 9 - Run the database migrations

This creates the tables in PostgreSQL:

```
alembic upgrade head
```

Step 10 - Start the server

Run this command and then open <http://localhost:8000> in your browser:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

You can register an account, create a chat, and start sending messages straight away. The interactive API documentation is available at <http://localhost:8000/docs>.

7. Security

Password storage

Passwords are hashed using Argon2, a memory-hard algorithm that makes brute-force attacks expensive even with GPU hardware. The plain-text password is never stored or written to any log.

JWT storage

Both tokens are stored in httponly cookies with SameSite=Lax. This means JavaScript running in the page cannot read them, which protects against XSS attacks.

Session control

Because session IDs live in Redis, the server can cancel any session instantly by deleting the key. Logout works immediately without waiting for the token to expire. Token rotation on refresh means an old refresh token cannot be reused.

GitHub OAuth

Only the authorization code flow is used. The GitHub client secret never leaves the server. The GitHub access token is used only to read the user profile and is not saved anywhere.

Input validation

Every request body is validated by Pydantic v2 before it reaches the service layer. SQLAlchemy uses parameterized queries everywhere, which prevents SQL injection.